

Open Benchmarks and Data Generation for LLM-Assisted RTL and Design Automation

Zhiyao Xie

Hong Kong University of Science and
Technology (HKUST)

July 2026



My Website



My GitHub

A Brief Self Intro of Zhiyao Xie

- Assistant Professor, ECE Department, **HKUST**, 2022 – Now
- Ph.D. in ECE, **Duke University**, 2017 – 2022
 - Supervisor: Yiran Chen, Hai (Helen) Li
- Research Interest: **EDA, AI-Assisted Chip Design**
- Selected Awards:
 - ACM SIGDA Outstanding New Faculty Award, 2026
 - ASPLOS Best Paper Award, 2026
 - HK RGC Early Career Award, 2023
 - ASPDAC Best Paper Award, 2023
 - ACM Outstanding Ph.D. Dissertation Award in EDA, 2023
 - EDAA Outstanding Ph.D. Dissertation Award, 2023
 - WAIC Yufan Award - Brilliant Star, 2023
 - MICRO Best Paper Award, 2021



Our explorations in open dataset & benchmark

- RTL generation benchmarking
 - RTL generation training dataset
 - RTL verification benchmarking
 - Synthetic dataset
- LLM for RTL design

Our explorations in open dataset & benchmark

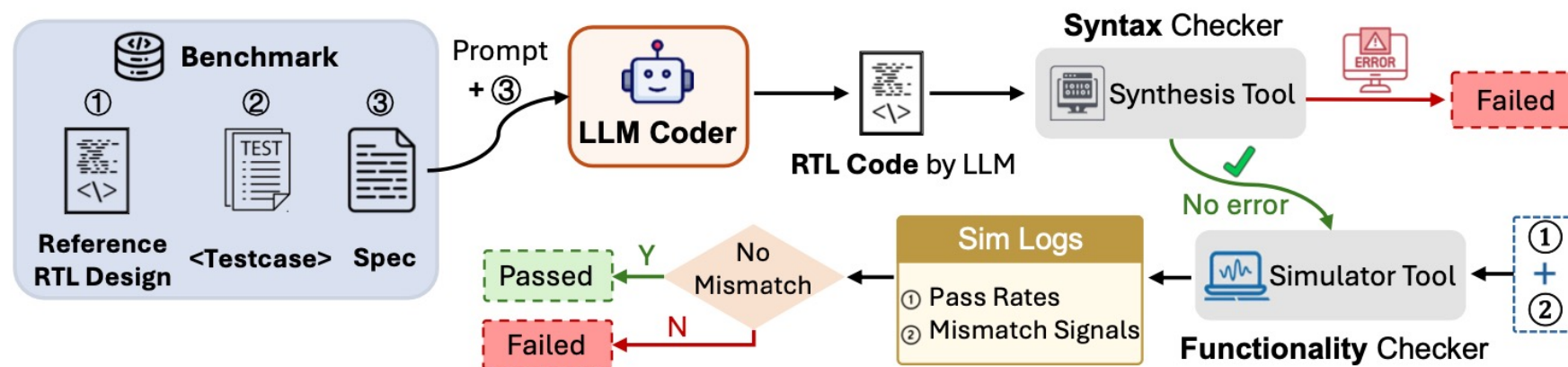
- RTL generation benchmarking
 - RTL generation training dataset
 - RTL verification benchmarking
 - Synthetic dataset
- LLM for RTL design

Benchmarking LLM for RTL Generation

[ASPDAC'24] RTLLM: one of 1st benchmarks evaluating RTL generation
(**highest cited (350+)** among all ASPDAC papers since 2021)

Benchmarks for RTL Code Generation			
Benchmarks	Open-sourced	link	Date
RTLLM [12, 154]	✓	https://github.com/hkust-zhiyao/rllm	2023-10
VerilogEval [14]	✓	https://github.com/NVlabs/verilog-eval	2023-12
VerilogEval v2[165]			2024-08

RTLLM established a standard evaluation flow



RTLLM2.0

[ICCAD'24]

50 design problems

Four categories

- 1. Arithmetic Modules
- 2. Memory Modules
- 3. Control Modules
- 4. Miscellaneous Modules

Arithmetic Modules		Memory Modules	
Design	Description	Design	Description
adder_8bit	An 8-bit adder	asyn_fifo	An asynchronous FIFO 16×8 bits
adder_16bit	A 16-bit adder implemented with full adders	LIFObuffer	A Last-In-First-Out buffer for temporary data storage
adder_32bit	A 32-bit carry-lookahead adder	right_shifter	Right shifter with 8-bit delay
adder_pipe_64bit	A 64-bit ripple carry adder based on 4-stage pipeline	LFSR	A Linear Feedback Shift Register for generating pseudo-random sequences
adder_bcd	A BCD adder for decimal arithmetic operations	barrel_shifter	A barrel shifter for rotating bits efficiently
sub_64bit	A 64-bit subtractor for high-precision arithmetic	RAM	8x4 bits true dual-port RAM
multi_8bit	An 8-bit multiplier based on shifting and adding operation	ROM	A Read-Only Memory module for storing fixed data
multi_16bit	A 16-bit multiplier based on shifting and adding operation	Miscellaneous Modules	
multi_booth_8bit	An 8-bit booth-4 multiplier	Design	Description
multi_pipie_4bit	A 4-bit unsigned number pipeline multiplier	clkgenerator	A clock generator for providing timing signals
multi_pipie_8bit	An 8-bit unsigned number pipeline multiplier	instr_reg	An instruction register module for holding and processing CPU instructions
div_16bit	A 16-bit divider based on subtraction operation	signal_generator	Generate various signal patterns
radix2_div	An 8-bit radix-2 divider	square_wave	A generator for producing square wave signals
comparator_3bit	A 3-bit comparator for comparing binary numbers	alu	An ALU for 32bit MIPS-ISA CPU
comparator_4bit	A 4-bit comparator for comparing binary numbers	pe	A Multiplying Accumulator for 32bit integer
accu	Accumulates 8-bit data and output after 4 inputs	freq_div	Frequency divider for 100M input clock, outputs 50MHz, 10MHz, 1MHz
fixed_point_adder	A fixed-point adder for arithmetic operations with fixed precision	freq_divbyeven	Frequency divider that divides input frequency by even numbers
fixed_point_subtractor	A fixed-point subtractor for precise fixed-point arithmetic	freq_divbyodd	Frequency divider that divides input frequency by odd numbers
float_multi	A floating-point multiplier for high-precision calculations	freq_divbyfrac	Frequency divider that divides input frequency by fractional values
Control Modules		calendar	Perpetual calendar with seconds, minutes, and hours
Design	Description	traffic_light	Traffic light system with three colors and pedestrian button
fsm	FSM detection circuit for specific input	width_8to16	First 8-bit data placed in higher 8-bits of the 16-bit output
sequence_detector	Detect specific sequences in binary input	synchronizer	Multi-bit mux synchronizer
counter_12	Counter module counts from 0 to 12	edge_detect	Detect rising and falling edges of changing 1-bit signal
JC_counter	A 4-bit Johnson counter with specific cyclic state sequence	pulse_detect	Extract pulse signal from the fast clock and create a new one in the slow clock
ring_counter	An 8-bit ring counter for cyclic state sequences	parallel2serial	Convert 4 input bits to 1 output bit
up_down_counter	A 16-bit counter that can increment or decrement based on control signals	serial2parallel	1-bit serial input and output data after receiving 6 inputs

Rethink: Any Problems in Existing Benchmarks?

Design Description

```
module binary_to_gray (
    parameter WIDTH = 6
)
```

Revised Description

```
module binary_to_gray #(
    parameter WIDTH = 6
)
```

Ambiguity analysis

A syntax error is in the parameter configuration of the sample code.

(a) Case “*binary_to_gray*” in CVDP benchmark

Design Description

Consider the Karnaugh map below.

```
x[1]x[2]
x[3]x[4]  00  01  11  10
00 | d | 0 | d | d |
01 | 0 | d | 1 | 0 |
11 | 1 | 1 | d | d |
10 | 1 | 1 | 0 | d |
```

Revised Description

Consider the Karnaugh map below.

```
x[0]x[1]
x[2]x[3]  00  01  11  10
00 | d | 0 | d | d |
01 | 0 | d | 1 | 0 |
11 | 1 | 1 | d | d |
10 | 1 | 1 | 0 | d |
```

Ambiguity analysis

The reference code is:

```
module RefModule (
    input [3:0] x,
```

.....
The input specified in the design description is `x[4:1]`, while in the testbench, the input is `x[3:0]`.

(b) Case “*113-2012_q1g*” in VerilogEval V1 benchmark

Design Description

The module should implement the FSM described by the state diagram:

```
A    --r0=0, r1=0, r2=0--> A
A    -----r0=1-----> B
A    -----r0=0, r1=1----> C
A    --r0=0, r1=0, r2=0--> D
B (g0=1) -----r0=1-----> B
B (g0=1) -----r0=0-----> A
C (g1=1) -----r1=1-----> C
C (g1=1) -----r1=0-----> A
```

Augmented Description

The module should implement the FSM described by the state diagram:

```
A    --r0=0, r1=0, r2=0--> A
A    -----r0=1-----> B
A    -----r0=0, r1=1----> C
A    --r0=0, r1=0, r2=0--> D
B (g0=1) -----r0=1-----> B
B (g0=1) -----r0=0-----> A
C (g1=1) -----r1=1-----> C
C (g1=1) -----r1=0-----> A
D (g1=1) -----r2=1-----> D
D (g1=1) -----r2=0-----> A
```

Mismatch analysis

In the testbench, the reference code contains the transition condition for state D:

```
case (state)
```

```
.....
```

```
C: next = r[1] ? C : A;
```

```
D: next = r[2] ? D : A;
```

```
.....
```

However, in the design description, transition from **D** is not specified in the design description, making it impossible to implement the logic based on the design description.

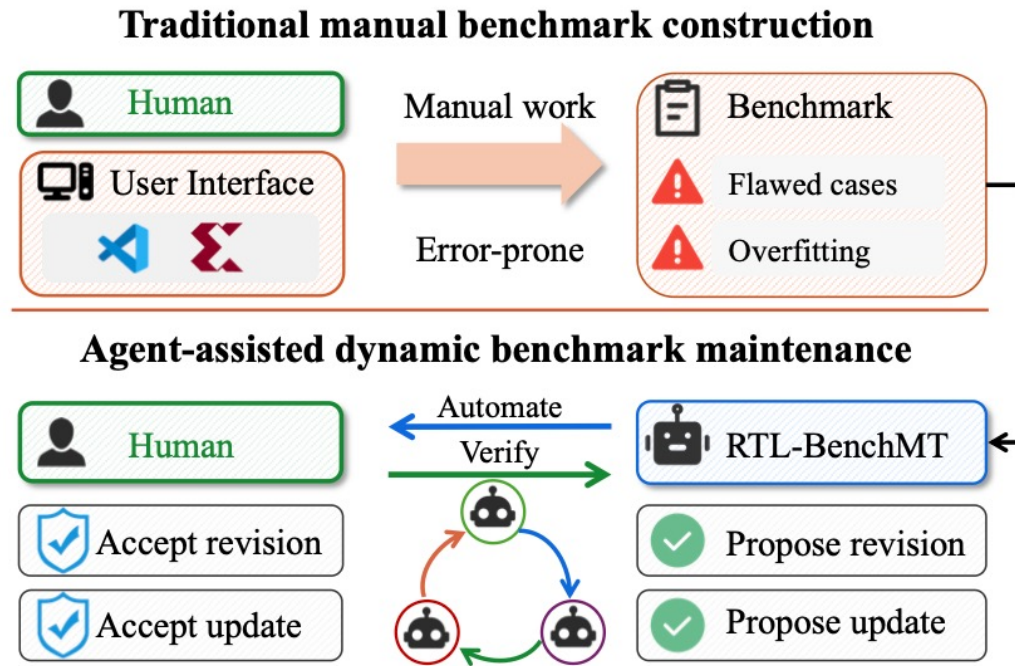
(c) Case “*148-q2afsm*” in VerilogEval V2 benchmark

Inconsistency between *instruction* and *code label*!!

- [DAC’26] Can we identify benchmarks mistakes and overfitting?

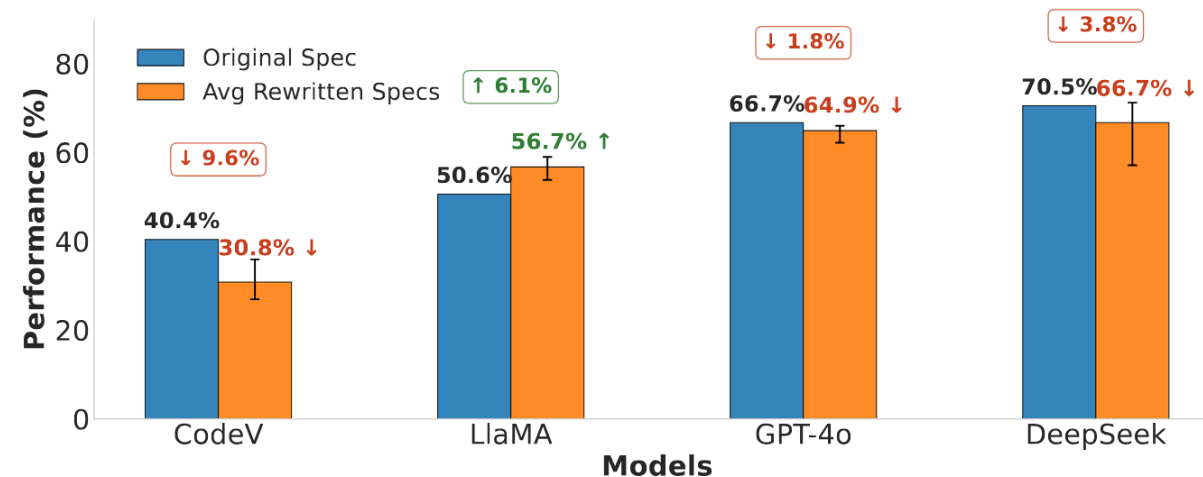
Benchmark Maintenance: Identify Error and Overfitting

BenchMT [DAC'26]: Benchmark maintenance



- (1) Automatically identifying and revising **flawed cases**
- (2) Automatically detecting **overfitting cases**

Overfitting detection by revising descriptions



Accuracy drop indicates potential overfitting

RTL-BenchLS: Scaling up Benchmarks

- RTL-BenchLS provides an **unprecedented scale in benchmark** with over 10,000 functionally verified Verilog designs.
- It introduces 3 tasks: (1) language-RTL round-trip reasoning; (2) masked-content reasoning; (3) project-level bug fixing

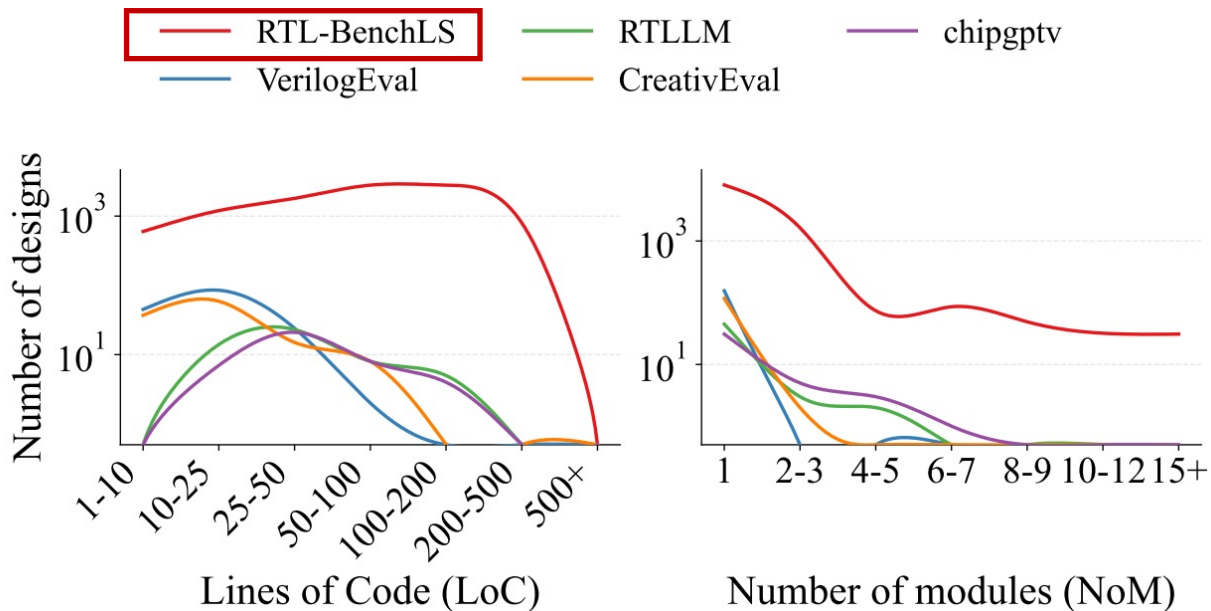


Table 1: Comparison of RTL benchmarks.

Benchmark	# Des.	Avg. LoC	Max. LoC	No Manual	Co-design	LEC/SEC
VerilogEval [17]	156	16	64			
RTLLM [20]	50	48	175			
CreativeEval [8]	119	19	98			
RTL-Repo [‡] [1]	4k+	18	260	✓		
ChipGPT-V [5]	40	49	171			
CVDP [†] [27]	783	—	—			
RTL-BenchLS	10k+	94	495	✓	✓	✓

Our explorations in open dataset & benchmark

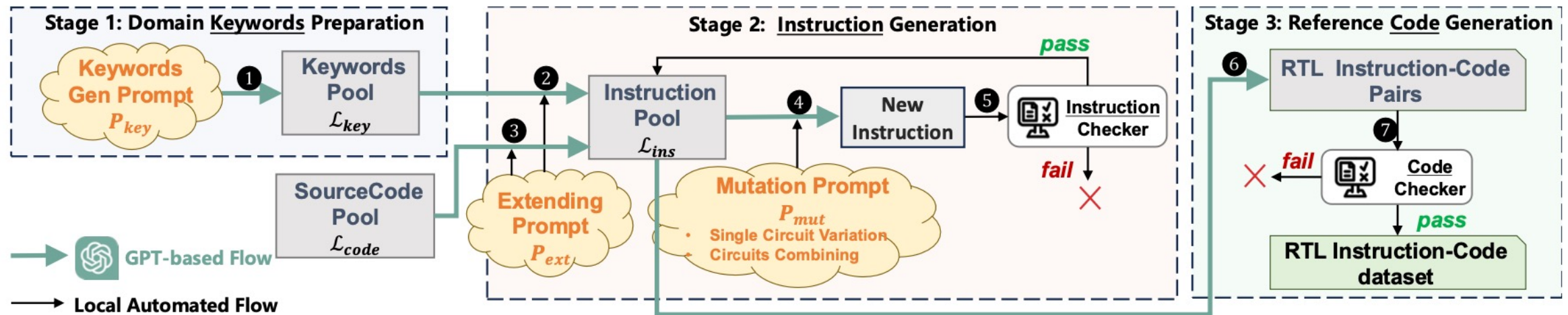
- RTL generation benchmarking
 - RTL generation training dataset
 - RTL verification benchmarking
 - Synthetic dataset
- LLM for RTL design

Early LLM Comparable to GPT-4 + Open Circuit Dataset

A major challenge for building LLMs: How to get the **dataset**?

RTLCoder [LAD'24 + TCAD'25] (300+ cited) proposes a data generation flow

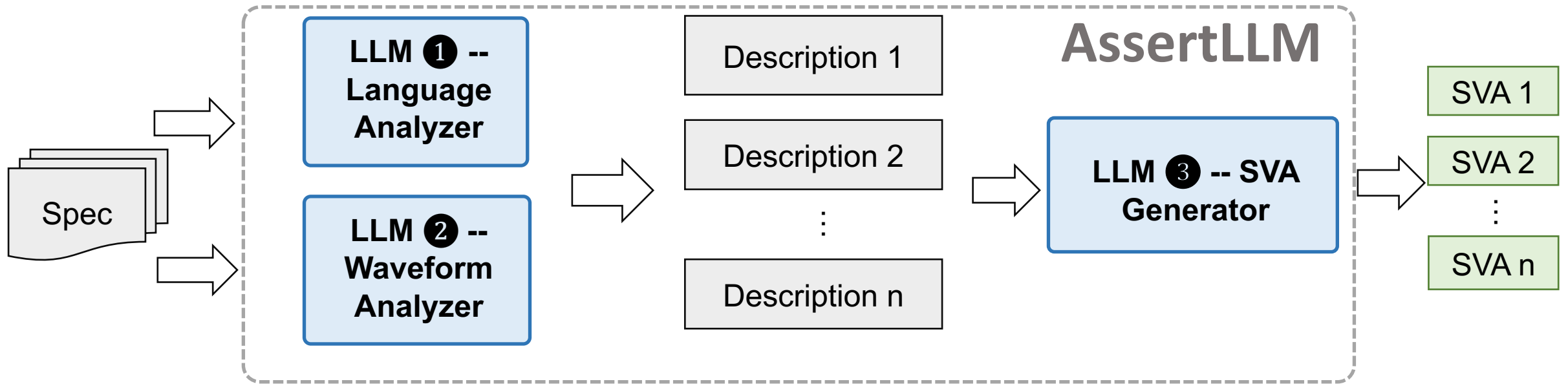
1. Generate & release 1st dataset with 27K **instruction-code pairs**
2. Fine-tune 1st open **LLMs** comparable to GPT-4 in RTL generation



Our explorations in open dataset & benchmark

- RTL generation benchmarking
 - RTL generation training dataset
 - RTL verification benchmarking
 - Synthetic dataset
- LLM for RTL design

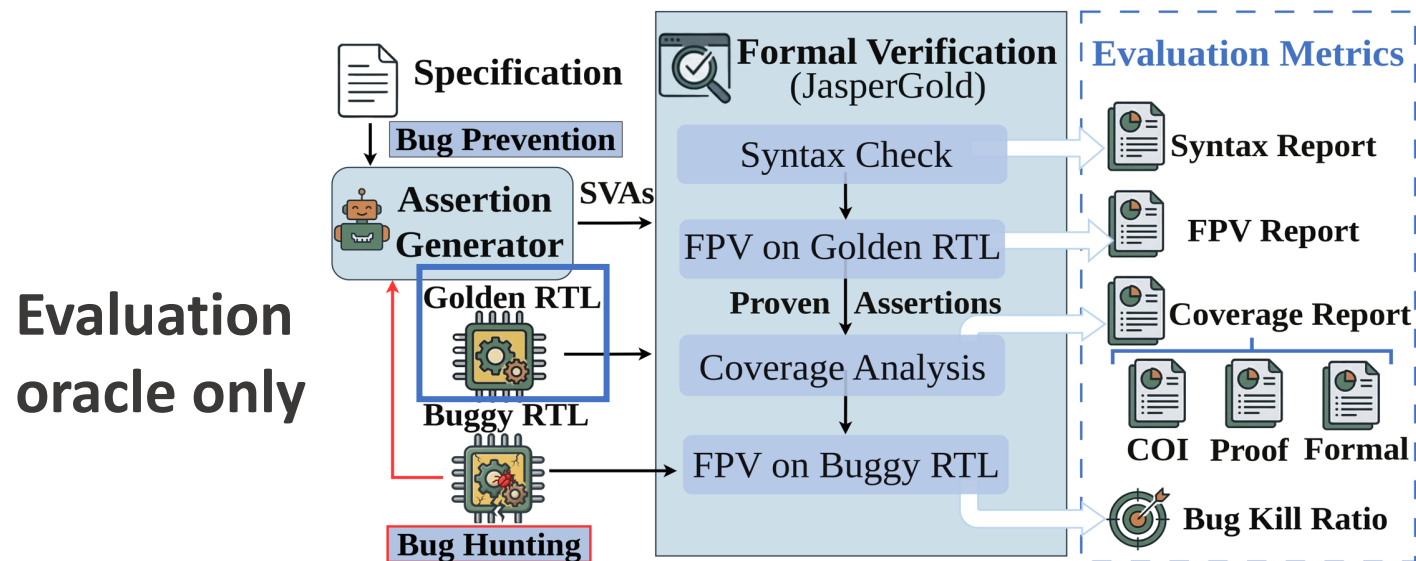
AssertLLM [ASPdac'24]



- A pioneering work [**cited 200+**] on generating **assertions** from **Spec**
- One of the first benchmarks, providing 20 designs and 3 metrics:
 - **Metric:** SVA ([SystemVerilog Assertions](#)) syntax, FPV pass/fail, COI coverage
 - **Designs:** Crypto units, processor cores, arithmetic units, protocols, ...

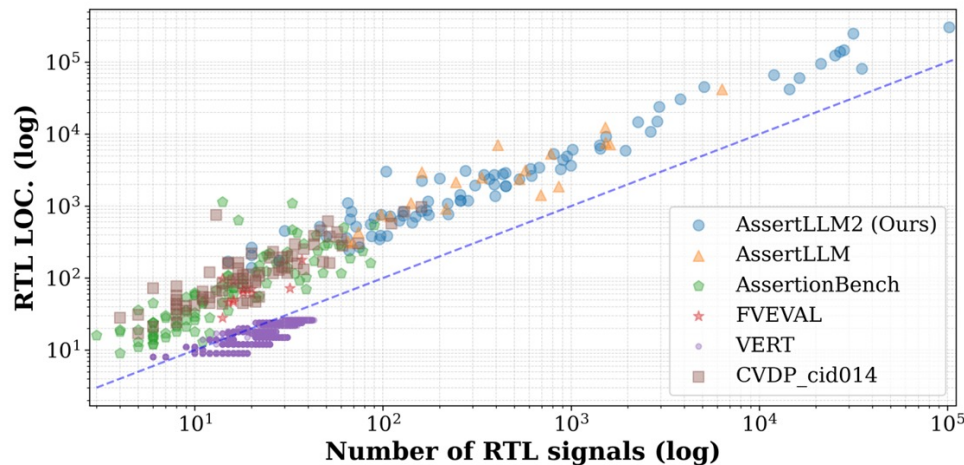
AssertLLM2 [ICCAD'26]: New Benchmark

- Two practical scenarios:
 - **Bug Prevention:** SPEC-only generation during RTL development.
 - **Bug Hunting:** SPEC + buggy RTL for exposing implementation errors.
- Richer evaluation:
 - Syntax, FPV, coverage, and mutation-based bug detection.



AssertLLM2 [ICCAD'26]

- 83 real-world designs, each with:
 - Markdown & pdf Specification
 - Golden RTL
 - 20 Buggy RTL Variants
- Diverse functionality
- More complex and real designs



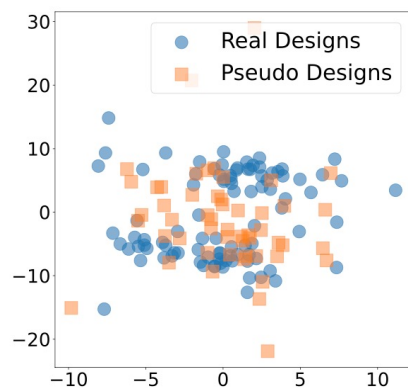
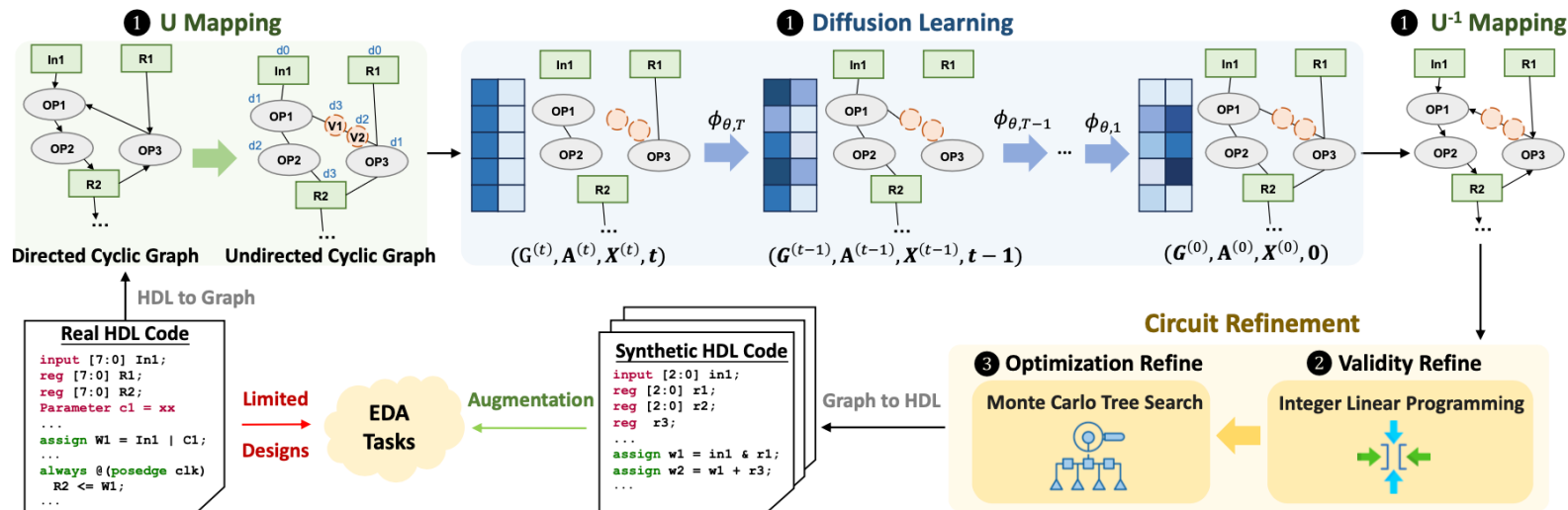
Category	Group	Brief Description	Avg. LOC
Arithmetic Core	arithmetic datapath	Datapath designs for arithmetic computation.	619
	trigonometric FPU	Hardware trigonometric computation unit.	838
	graphics accelerator	Arithmetic engine for graphics workloads.	3267
	arithmetic utilities	Counter, noise, and decoding arithmetic logic.	571
Communication Controller	Ethernet MAC	Ethernet frame transmit/receive MAC.	3716
	i ² c	I ² C serial bus controllers and host bridges.	1250
	IEEE 1588	Precision timestamping and clock-sync logic.	1386
	SPI	SPI flash, master, and protocol-bridge controllers.	1282
	UART	UART peripherals and UART-centric bus bridges.	605
	SATA phy	SATA physical-layer interface logic.	2727
	SD/MMC host controller	SD/MMC command and data host controllers.	2788
Coprorocessor	sport interface	Synchronous serial port controller.	588
	xgate	Auxiliary task-offload coprocessor core.	2886
Crypto Core	AES	AES-family symmetric encryption engine.	796
	pairing-based crypto core	Pairing-oriented cryptographic computation core.	1486
	lightweight security core	Lightweight ciphers and hash accelerators.	421
DSP Core	IMA adpcm codec	IMA ADPCM audio codec engine.	237
	PID controller	Digital proportional-integral-derivative controller.	1935
	FFT processor	Fast Fourier transform processor.	753
	ECG signal chain	ECG acquisition, filtering, and extraction datapath.	1422
ECC Core	reed-solomon decoder	Reed-Solomon error-correction decoder.	3305
	gf256 rs core	GF(256)-based Reed-Solomon codec core.	453
Library	utility library	Reusable RTL utility components.	381
Memory Core	SDram controller	External SDRAM access controller.	2280
	memory buffer	On-chip RAM and FIFO buffering logic.	1189
Processor	embedded CPU	Embedded general-purpose processor cores.	3415
	RISC CPU	Modern embedded RISC processor core.	10312
	legacy-compatible CPU	Legacy ISA-compatible processor cores.	19610
	educational CPU	Teaching-oriented simplified processor core.	627
	graphics processing unit	Parallel graphics compute engine.	6068
System Controller	system controller family	Memory controller and PCI bridge logic.	9225
System-on-Chip	Wishbone SOC IP family	Wishbone SoC interconnect and DMA backbone.	6438
Video Controller	display compositor	Text and sprite display compositor.	962
	video display pipeline	Video timing and pixel output pipeline.	1602
Other	AC97 controller	AC'97 audio controller.	1897
	utility modules	Miscellaneous low-complexity support RTL.	372
	timebase controller	Timer and PWM generation peripherals.	380

Our explorations in open dataset & benchmark

- RTL generation benchmarking
 - RTL generation training dataset
 - RTL verification benchmarking
 - Synthetic dataset
- LLM for RTL design
- 

Generation of Synthetic Circuits

- Generate **synthetic circuits**: SynCircuit [DAC'25], Sync-LLM [EMNLP'25]



Synthetic designs are similar to real designs

Timing Path Group 'clk' (max_delay)

Level of Logic: 43
 Critical Path Length: 3.264
 Critical Path Slack: -2.732
 No. of Violating Paths: 3867

Cell & Pin Count

Pin Count: 640145

Leaf Cell Count: 173466

Synthetic designs reach >100K cells

- Synthetic circuit generation based on graph generation models**
- Synthetic circuits enable “big data”**

	Target	R	MAPE	RRSE
No Pseudo-Circuits	WNS	NA	52 %	2.1
GraphRNN [27]		0.71	42 %	1.7
DVAE [28]		0.75	77%	2.6
CircuitGen		0.88	36 %	1.3

Pseudo-designs can boost AI accuracy in IC prediction

[1] SynCircuit: Automated Generation of New Synthetic RTL Circuits Can Enable Big Data in Circuits, **DAC, 2025**

[2] SynC-LLM: Generation of Large-Scale Synthetic Circuit Code with Hierarchical Language Models, **EMNLP (main), 2025**

Summary

We explored open dataset & benchmark in various aspects

- LLM for RTL design
 - RTL generation benchmarking
 - RTL generation training dataset
 - RTL verification benchmarking
- Open dataset & synthetic dataset



My Website



My GitHub

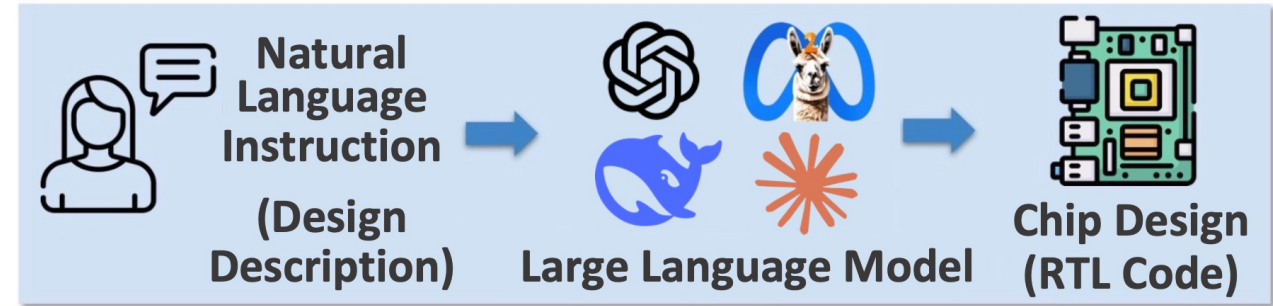
Benchmarks shape a field; **Datasets** are the foundation of AI methods

Backup Slides

Generative AI: LLM for RTL Generation & More

Task: LLM-based RTL Generation

- Input: **Design description**
 - Describe target functionality.
 - Module names, I/O names.
- Output: **Design in RTL code**

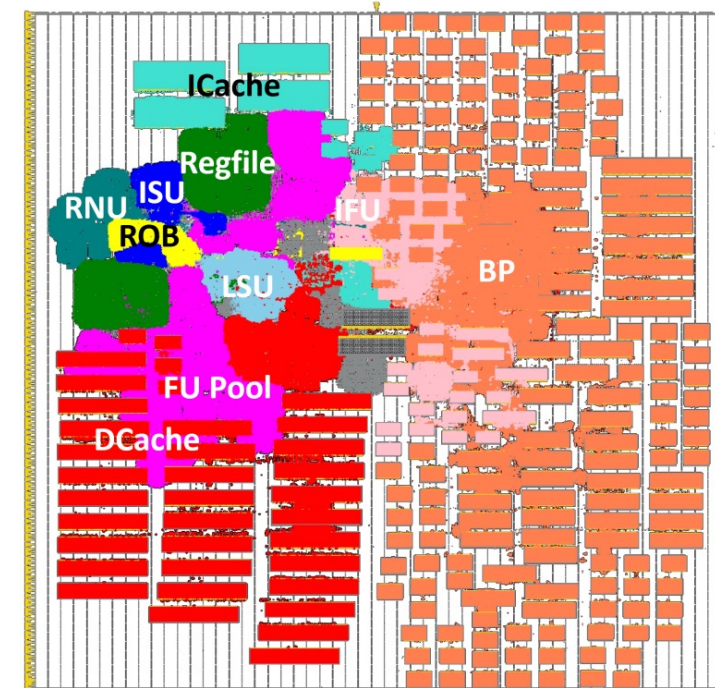
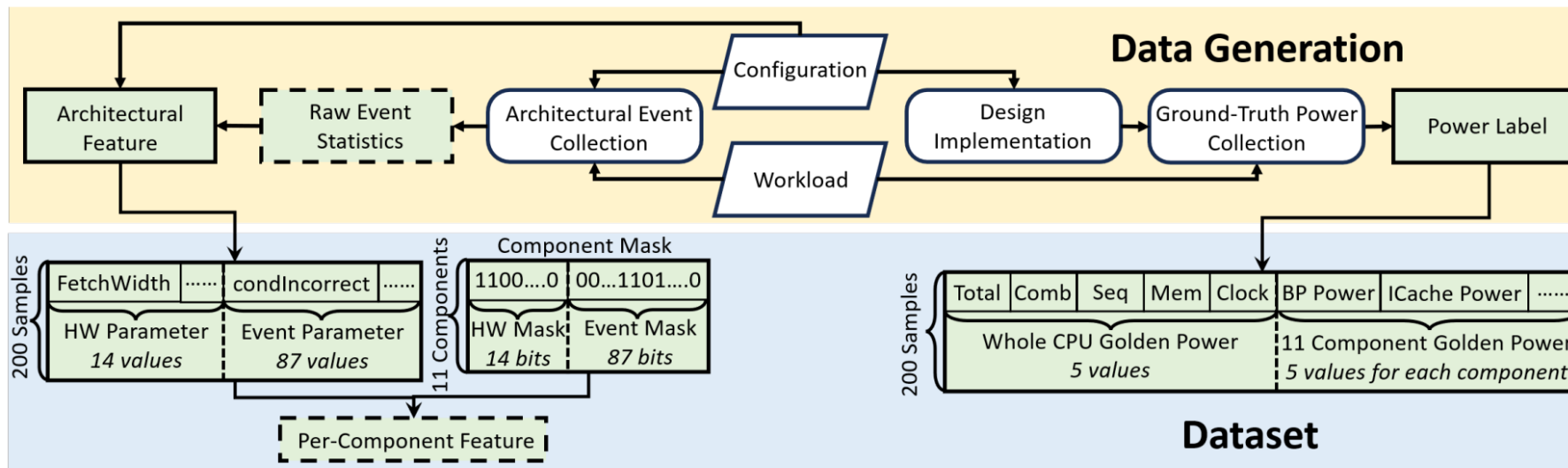


In addition to hardware code generation:

- LLM for hardware code **optimization, debugging, verification, ...**

1. Open Circuit Dataset & Benchmark

- Dataset for Arch-Level CPU Power Modeling: **ArchPower [NeurIPS'25]**
- Provide fine-grained power labels: from 25 different CPU configurations when executing 8 different workloads.



[1] ArchPower: Dataset for Architecture-Level Power Modeling of Modern CPU Design, **NeurIPS D&B Track, 2025.**

Existing Benchmarks Do Not Reflect Real Verification

- Unrealistic task formulation
 - **Verified golden RTL** is often provided as model input.
- Weak benchmark inputs
 - Specifications are simplified; designs are small or limited in number.
- Oversimplified evaluation: very limited metrics

Works	Task		SPEC Format	Dataset			Evaluation Metric					
	Bug Preventing	Bug Hunting		# Designs	Sample Scale	Avg. LOC.	Syntax	Coverage			FPV Result	Mutation-Based Bug Detection
VERT	-	✓*	-	20000	Snippet	18	✓	✓	-	-	✓	✓
AssertionBench	-	✓*	-	101	Module	123	✓	-	-	-	✓	-
FVEval	✓	✓*	Partial Sentences	379	Snippet	73	✓	-	-	-	✓	-
CVDP	-	✓*	Pre-extracted Checklists	98	Module/IP	167	✓	-	-	-	-	-
AssertLLM	✓	-	Unstructured Raw PDFs	20	IP/System	5146	✓	✓	-	-	✓	-
Ours	✓	✓	Structured & Unified SPEC	83	IP/System	2888	✓	✓	✓	✓	✓	✓

✓*: use the golden RTL during assertion generation